ISOLATION LEVELS IN MYSQL / MARIADB

CONSISTENCY / CONCURRENCY / PERFORMANCE

Different levels.

Different trade-offs.

Reliable Data.

JAMES MCGUIGAN



02

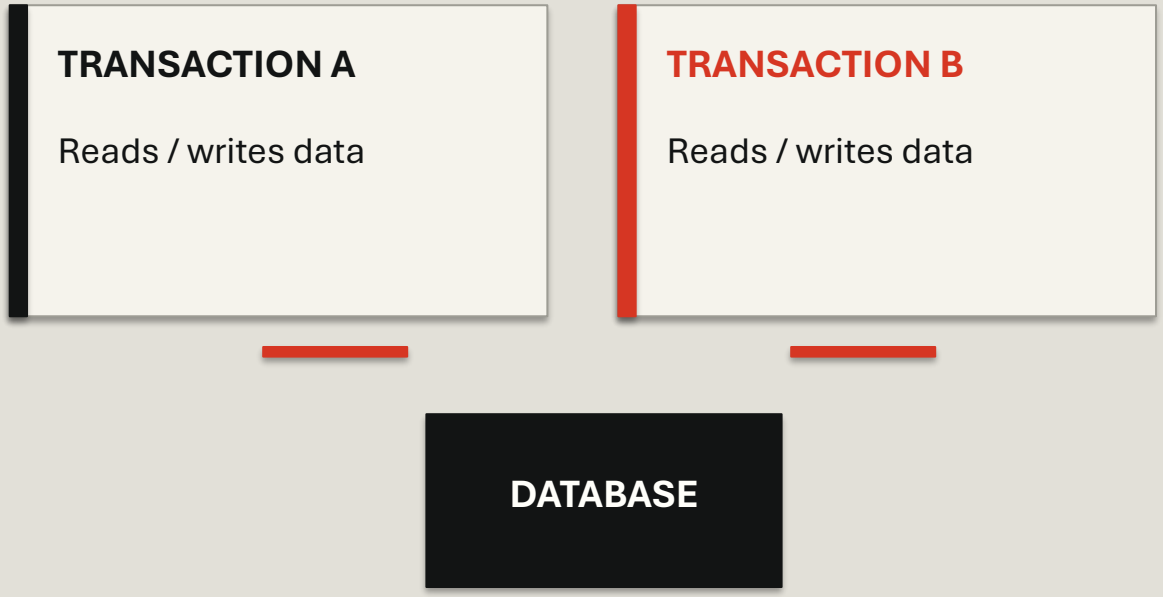
THE BIG PICTURE

WHAT IS AN ISOLATION LEVEL?

DEFINITION

An isolation level controls how transactions see and interact with each other's data.

It determines what kinds of temporary or inconsistent reads are allowed.



WHY IT MATTERS

- Prevents surprising data anomalies
- Balances consistency and throughput
- Helps applications behave predictably

03

COMMON ANOMALIES

READ PHENOMENA & RISK

DIRTY READ

A transaction reads another transaction’s uncommitted change. If the writer rolls back, the value was never committed.

NON-REPEATABLE READ

A transaction reads the same row twice and gets different values because another transaction committed an update in between.

PHANTOM READ

The same search returns a different set of rows because another transaction inserted or deleted rows matching the condition.

LOST UPDATE

Two transactions change the same record; one write can overwrite the other. This is a related concurrency risk, not one of the three classic read phenomena.

04

THE FOUR ISOLATION LEVELS

INNODB BEHAVIOR QUERY TYPE AND LOCKING STRATEGY MATTER

LEVEL	DIRTY READ	NON-REPEATABLE	PHANTOM READ	TRADE-OFF
READ UNCOMMITTED	Possible	Possible	Possible	Highest concurrency; weakest consistency
READ COMMITTED	Prevented	Possible	Possible	Fresh committed view per read
REPEATABLE READ	Prevented	Prevented for snapshot reads	Depends on read type; locking range reads use gap/next-key locks	MySQL InnoDB default
SERIALIZABLE	Prevented	Prevented	Prevented through stricter serialization/locking	Strongest protection; more waiting

05

ANOMALIES IN ACTION

WHAT TWO CONCURRENT TRANSACTIONS CAN DO

DIRTY READ EXAMPLE

T1: UPDATE accounts SET balance = 1000; (not committed)
T2: SELECT balance → sees 1000
T1: ROLLBACK

T2 saw a value that never became permanent.

NON-REPEATABLE READ

T1: SELECT balance → 500
T2: UPDATE balance = 700; COMMIT
T1: SELECT balance → 700

Same row, different result inside T1.

PHANTOM READ EXAMPLE

T1: SELECT COUNT(*) FROM orders WHERE total > 100 → 3
T2: INSERT order with total = 150; COMMIT
T1: run the same search → may see 4 under weaker isolation.

LOST UPDATE EXAMPLE

Two shoppers read stock = 1. Both decide to decrement it. If updates are not coordinated, one write may overwrite the other's result.

Use appropriate locking or atomic updates.

06

WHY MULTIPLE LEVELS?

NO SINGLE SETTING IS BEST FOR EVERY WORKLOAD

01

PERFORMANCE VS. CONSISTENCY

Higher isolation can mean more locking, waiting, and deadlocks. Lower isolation can improve throughput but allow more inconsistent observations.

02

DIFFERENT USE CASES

A dashboard may tolerate changing counts. Inventory checkout and financial transfers need stronger correctness guarantees.

03

PRACTICAL TRADE-OFF

Choose the least restrictive level that meets the correctness requirement, then test concurrent workloads.

04

ENGINE DETAILS MATTER

MySQL/MariaDB InnoDB use MVCC and locking strategies. Exact behavior depends on isolation level and read type.

HIGHER ISOLATION = MORE PROTECTION

LOWER ISOLATION = MORE FLEXIBILITY

07

KEY TAKEAWAYS

A PRACTICAL WAY TO THINK ABOUT ISOLATION

01 Isolation defines what each transaction can see, and when.

02 Each level changes the risk of dirty reads, non-repeatable reads, and phantoms.

03 Multiple levels exist to balance correctness, throughput, and workload needs.

EXAMPLE SQL

```
-- Set for the next transaction
SET TRANSACTION ISOLATION LEVEL
REPEATABLE READ;

START TRANSACTION;
SELECT balance FROM accounts WHERE id = 1;
COMMIT;
```

Start with InnoDB's default REPEATABLE READ, then change it only when the workload and tests justify it.